

Introduction to Git

- **Version Control Definition:** A system to manage changes in files, allowing tracking, combining versions, and reverting changes.
- **Importance:** Ensures consistency and reliability in projects by enabling easy reversion to previous versions if issues arise.
- **Git Overview:** An open-source version control system that tracks changes, compares file contents over time, and identifies who made changes and when.
- **Basic Shell Commands:**
 - `pwd`: Prints the current working directory.
 - `ls`: Lists files and directories in the current location.
 - `cd [directory]`: Changes the current directory.
- **Checking Git Version:**
`git --version`

Creating Repos

- **Creating a New Git Repository:**

A repository (repo) consists of your project files and directories, along with Git's extra information about the project's history.

To create a new Git repo, use the command:

```
git init mental-health-workspace
```

This command initializes a new repo in the specified directory.

- **Checking Repo Status:**

After initializing a repo, you can check its status with:

```
git status
```

This command confirms the repo's initialization and suggests tracking new files with `git add`.

- **Converting an Existing Project to a Git Repo:**

If you have an existing project, you can convert it to a Git repo by running:

```
git init
```

This command initializes a repo in the current directory and recognizes existing files that are not yet tracked.

- **Avoiding Nested Repos:**

Generally, avoid creating a Git repo inside another Git repo to prevent confusion with multiple `.git` directories.

Staging and Committing files

- **Staging Area:** This is where Git tracks modified files before they are committed. Think of it as placing a letter in an envelope.
- **Committing Files:** Making a commit in Git is like putting the envelope in a mailbox. It saves a snapshot of your files at that point in time.

Commands:

Adding a File to the Staging Area:

```
git add report.md
```

- This command adds report.md to the staging area.

Adding All Modified Files:

```
git add .
```

- This command stages all modified files in the current directory and sub-directories.

Committing Files:

```
git commit -m "Your log message here"
```

- This command commits the staged files with a log message.

Comparing Versions

Understanding Diffs: A diff shows the differences between two file versions. The `git diff` command compares the last committed version with the modified version not yet staged.

```
git diff report.md
```

- **Interpreting Diff Output:** The output indicates changes with lines starting with `-` (removed) and `+` (added). For example, `-` in red text represents a line in the previous version that is not in the current version.

Comparing Staged Files: Use the `--staged` flag to compare the staged version with the last committed version.

```
git diff --staged mental_health_survey.csv
```

- **Comparing Commits:** Use commit hashes or HEAD references to compare specific commits. For example, to compare the most recent commit with the second most recent:

```
git diff HEAD~1 HEAD
```

- **git revert:** This command undoes changes by creating a new commit that reinstates previous versions of files. For example, to revert the last commit without opening a text editor, you use:

```
git revert HEAD --no-edit
```

- **git checkout:** This versatile command allows you to revert a single file to a previous state. For instance, to revert a file to its state in the second-most recent commit:
`git checkout HEAD~1 -- filename`
- **git restore:** This command unstages files, moving them back to the working directory. To unstage a specific file:
`git restore --staged filename`